GUIDE 05

Connect Spring Boot to MySQL and Configure Environment Variables

UniMart Monolithic Full-Stack Lab Series

B.Sc. (Hons) IT - Level 2 | Hands-on Laboratory Practical

Department of Industrial Management
Faculty of Science, University of Kelaniya

Learning outcomes

- Connect the backend to MySQL using environment-driven Spring configuration.
- Use Flyway migrations and Hibernate validation safely.
- Separate local, test and production profiles.
- Create JWT and CORS configuration without committing secrets.

Prerequisites

- Guides 03 and 04 completed.
- The unimart schema and unimart_app account exist.

Suggested student time: 2-2.5 hours

1. Required environment variables

Variable	Local example	Purpose
DB_URL	jdbc:mysql://127.0.0.1:3306/unimart	JDBC connection URL.
DB_USERNAME	unimart_app	Least-privilege database account.
DB_PASSWORD	local password	Secret; never commit.
JWT_SECRET	base64-encoded random 32+ byte value	Signs and verifies HS256 access tokens.
JWT_ACCESS_MINUTES	15	Short access-token lifetime.
APP_ALLOWED_ORIGINS	http://localhost:5173	Exact frontend origin list.
SERVER_PORT	8080	Backend port.
SPRING_PROFILES_ACTIVE	local	Selects profile.

2. Create safe example files

```
[Environment template]
# .env.example - commit names and safe placeholders only
DB_URL=jdbc:mysql://127.0.0.1:3306/unimart
DB_USERNAME=unimart_app
DB_PASSWORD=change-me
JWT_SECRET=replace-with-base64-secret
JWT_ACCESS_MINUTES=15
APP_ALLOWED_ORIGINS=http://localhost:5173
SERVER_PORT=8080
SPRING_PROFILES_ACTIVE=local
```

```
[Git ignore]
# .gitignore additions
.env
.env.*
!.env.example
application-secrets.yml
*.pem
*.key
```

```
[Generate JWT secret]
openssl rand -base64 48
```

3. Base Spring configuration

```yaml
[YAML]
# src/main/resources/application.yml
spring:
  application:
    name: unimart-backend
  profiles:
    default: local
  datasource:
    url: ${DB_URL}
    username: ${DB_USERNAME}
    password: ${DB_PASSWORD}
    hikari:
      maximum-pool-size: ${DB_POOL_SIZE:10}
      minimum-idle: ${DB_MIN_IDLE:2}
  jpa:
    open-in-view: false
    hibernate:
      ddl-auto: validate
    properties:
      hibernate:
        jdbc:
          time_zone: UTC
        format_sql: false
  flyway:
    enabled: true
    locations: classpath:db/migration

server:
  port: ${SERVER_PORT:8080}
  error:
    include-message: never

management:
  endpoints:
    web:
      exposure:
        include: health,info

app:
  security:
    jwt-secret: ${JWT_SECRET}
    access-minutes: ${JWT_ACCESS_MINUTES:15}
  cors:
    allowed-origins: ${APP_ALLOWED_ORIGINS:http://localhost:5173}
```

Schema ownership: Use ddl-auto: validate, not update, outside throwaway prototypes. Flyway creates and changes the schema; Hibernate checks that mappings match it.

4. First Flyway migration

```sql
-- src/main/resources/db/migration/V1__create_core_tables.sql
CREATE TABLE users (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  university_email VARCHAR(190) NOT NULL UNIQUE,
  password_hash VARCHAR(100) NOT NULL,
  full_name VARCHAR(120) NOT NULL,
  role VARCHAR(30) NOT NULL,
  email_verified BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
    ON UPDATE CURRENT_TIMESTAMP
);

CREATE TABLE categories (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(80) NOT NULL UNIQUE,
  active BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE listings (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  seller_id BIGINT NOT NULL,
  category_id BIGINT NOT NULL,
  title VARCHAR(160) NOT NULL,
  description TEXT NOT NULL,
  price DECIMAL(12,2) NOT NULL,
  status VARCHAR(30) NOT NULL,
  version BIGINT NOT NULL DEFAULT 0,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
    ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_listing_seller FOREIGN KEY (seller_id) REFERENCES users(id),
  CONSTRAINT fk_listing_category FOREIGN KEY (category_id) REFERENCES categories(id),
  CONSTRAINT chk_listing_price CHECK (price >= 0),
  INDEX idx_listing_search (status, category_id, created_at)
);
```

Create later migrations for orders and reviews rather than editing V1 after it has run on shared environments. A migration is an immutable history record.

5. JPA entity mapping example

```java
@Entity
@Table(name = "listings")
public class Listing {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;

  @ManyToOne(fetch = FetchType.LAZY, optional = false)
  @JoinColumn(name = "seller_id", nullable = false)
  private User seller;

  @Column(nullable = false, precision = 12, scale = 2)
  private BigDecimal price;

  @Enumerated(EnumType.STRING)
  @Column(nullable = false, length = 30)
  private ListingStatus status;

  @Version
  private Long version;

  // Other fields and generated getters/setters omitted.
}
```

6. JWT encoder/decoder configuration

```
[Java]
@Configuration
@EnableWebSecurity
public class SecurityConfig {

  @Bean
  SecretKey jwtSecretKey(@Value("${app.security.jwt-secret}") String value) {
    byte[] decoded = Base64.getDecoder().decode(value);
    if (decoded.length < 32) {
      throw new IllegalStateException("JWT secret must be at least 32 bytes");
    }
    return new SecretKeySpec(decoded, "HmacSHA256");
  }

  @Bean
  JwtEncoder jwtEncoder(SecretKey key) {
    return NimbusJwtEncoder.withSecretKey(key).build();
  }

  @Bean
  JwtDecoder jwtDecoder(SecretKey key) {
    return NimbusJwtDecoder.withSecretKey(key).build();
  }
}
```

The complete SecurityFilterChain is added in the CRUD/integration guides. Spring Security validates bearer JWTs; password authentication is performed at login before a token is minted.

7. Configure local variables in IntelliJ

1. Open Run > Edit Configurations > UniMartApplication.
2. Set SPRING_PROFILES_ACTIVE=local.
3. Add DB_URL, DB_USERNAME, DB_PASSWORD, JWT_SECRET and APP_ALLOWED_ORIGINS in the Environment variables field.
4. Do not paste secrets into VM options, source files, screenshots or shared instructions.
5. Run the application and inspect logs for Flyway success and Hikari connection-pool startup.

8. Verify the connection

```
[Terminal]
./mvnw spring-boot:run
curl http://localhost:8080/actuator/health
```

Expected health status is UP. In Workbench, refresh the unimart schema and verify flyway_schema_history and the migrated tables exist.

9. Error diagnosis

Message	Meaning	Fix
Access denied for user	Credentials/host/grants incorrect	Test the same account in Workbench; verify grants and environment values.
Unknown database	DB_URL schema does not exist	Create unimart and confirm spelling/case.
Communications link failure	Server stopped, port/firewall issue	Start MySQL; check 127.0.0.1:3306.
Flyway validation failed	Migration file changed after execution	Restore original migration; create a new versioned migration.
Schema-validation missing column	Entity and migration differ	Align mapping and SQL through a new migration.
JWT secret error	Not base64 or too short	Generate a new 48-byte base64 value and restart.

Message	Meaning	Fix
CORS blocked	Origin mismatch	Use exact protocol, host and port in APP_ALLOWED_ORIGINS.

Completion checklist

☐ Local secrets are provided as environment variables only.

☐ Flyway creates the schema and Hibernate validates it.

☐ Actuator health reports UP.

☐ Database tables are visible in Workbench.

☐ JWT secret is random, base64 encoded and at least 32 bytes after decoding.

☐ CORS origin is exact and not a wildcard.